Grok Bot and Alternatives

Last modified: 2026-08-29 22:10:28 (PDT)

Coding agents edit a checkout and return a branch or pull request (coding-agent platforms). *Grok Bot-style* products are a different shape: a named, persistent teammate with its own computer, so it can click through apps and websites the way a person would and keep going after you close your laptop. This chapter reviews Grok Bot and the alternatives we could verify from primary sources, including Rakazo.

The question for the lab is not "is the demo impressive?" but "what does it add to Claude Code plus `ai-config`, and what data or approval boundary does it move?"

> ⚠ A fast-moving, opinionated snapshot
>
> As of August 2026, this category is days to weeks old in public form. Grok Bot launched as an early beta on 11 August 2026 (xAI 2026d). Product names, plan gates, and security wording will drift. The claims below are taken from vendor docs and project READMEs as they stood on 26 August 2026. Re-check those sources before acting on any of them.

What This Category Is

A coding agent lives in a repository. A Grok Bot-style teammate lives on a *computer*: a browser, a filesystem, a terminal, and often a graphical desktop, with logins and files that survive from one task to the next.

That architecture buys three things coding agents do not optimize for:

- Work in tools that have no clean API or MCP server, by driving the user interface.
- Unattended runs after the operator's laptop is closed.
- Named roles that keep memory, routines, and preferences instead of starting from a fresh sandbox every issue.

It also moves a different risk: the agent holds app sessions, not just a git checkout. A login placed on a shared computer is available to every teammate on that computer. Vendor docs for Grok Bot are explicit that **named Bots are not a security boundary** (xAI 2026a).

This chapter is not a catalog of every desktop chat app. It covers products that either ship that "teammate with a computer" shape or are the closest vendor and self-hosted substitutes a lab member would actually reach for.

What We Already Use

The lab already runs agents for repository work, documented in the coding-agents chapter and the orchestration chapter. The pieces that matter as a yardstick here are:

- **Claude Code** (terminal, IDE, and cloud), with skills, hooks, permissions, and subagents.
- **Portable agent config** in `Morrison-Lab/ai-config`: skills, hooks, and memories that we install once and expect a session to load (how the config reaches a machine; customizing an agent).
- **GitHub as the review surface**: work returns as a branch or pull request, with human approval before merge.
- **Cursor** as an interactive IDE, not as a second cloud computer that signs into our apps.

A new teammate product is useful to us only if it does something that stack does not already do well, without putting lab credentials on a computer we do not control.

Grok Bot

Grok Bot (xAI 2026c) is xAI's early-beta product for named AI teammates. Each Bot is a persistent agent you message like a colleague. The Bots on one account share **one user-scoped cloud computer** with a browser, filesystem, and terminal (xAI 2026f). They can use connectors (shown as Plugins in the app) where those exist, and computer use for apps and websites without a clean API. Several Bots can run in parallel, each with its own *screen* on that shared computer. The screens are work surfaces, not isolation.

It is a separate product from the consumer Grok chatbot. You install a desktop app (macOS or Windows), sign in with a **Cursor** account, and can continue the same Bot from iOS (xAI 2026b). There is no Linux desktop app as of this survey. Eligible plans listed in the getting-started docs are:

- SuperGrok Plus and SuperGrok Heavy
- Cursor Pro+ and Cursor Ultra
- Cursor Teams Standard and Premium

Grok Bot requires cloud data storage. Accounts using Cursor Legacy Privacy Mode must change that setting before it will start (xAI 2026b). Privacy, training opt-out, and account deletion follow Cursor's privacy and security documentation, not a separate Grok-only data plane (xAI 2026a).

Skills, routines, and demonstration

Grok Bot has its own in-app skill and routine objects (xAI 2026e):

- A **skill** is a reusable instruction pack (when to use it, inputs, steps, validation, deliverable, approvals).
- A **routine** tells one Bot when to run a workflow, on a schedule or, where supported, after a Cursor-account event such as a Slack message or GitHub notification.
- **Teach a task**, when the control is visible, records a browser demonstration (up to ten minutes, no microphone) and drafts a skill from it.

A Bot can own up to 50 routines. Background routines can run while the laptop is closed. A test run performs *real* work.

Those objects are Grok Bot's product surface. They are not a substitute for the lab `ai-config` corpus. Because Grok Bot signs in with a Cursor account, a session can still load that corpus as a **Cursor plugin** (skills, user-global rules, and commands from `.cursor-plugin/plugin.json`): the same plugin path this lab's Cursor sessions already use. That is not Claude Code's install path. Grok Bot does not load the `~/.claude` symlink install, `CLAUDE.md` `@imports`, or Claude hooks/`hooks.json`. Cursor Cloud uses `.cursor/hooks.json` instead (how the config reaches a machine).

Approvals and the shared-computer boundary

The security docs describe operator controls worth taking at face value (xAI 2026a):

- Per-action **Allow once**, **Deny**, and **Always allow**.
- **Auto Review** rules: Require Approval always wins over Always Allow when both match.
- **Take control** of the Agent Computer for passwords, passkeys, two-factor codes, CAPTCHAs, and payments. Do not paste those into chat.
- Local-computer execution is a *separate* switch (default: ask every time) and does not stop the Bot from using its cloud computer.
- Deleting a Bot does not wipe shared-computer files or browser sessions.

The docs' own least-privilege advice matches lab practice: connect only the tools a workflow needs, start with drafts, and keep sending, publishing, purchasing, deletion, and production changes behind approval.

Useful to us? Not as the agent-config workflow

Grok Bot is a capable product in a category we do not currently run: unattended work inside signed-in web apps. It does not replace Claude Code. Plugin skills from `Morrison-Lab/ai-config` **do** load when the Cursor plugin is installed (`.cursor-plugin/plugin.json`: skills, user-global rules, commands). The Claude-only install path does **not**: no `~/.claude` symlink, no `CLAUDE.md` `@imports`, no Claude `hooks.json`. Cursor Cloud uses `.cursor/hooks.json` instead. It has no Linux desktop app. Every Bot on an account shares logins and files. Using it for lab GitHub, email, or data systems would put those sessions on an xAI-managed computer under Cursor's data settings.
If someone already has an eligible Cursor or SuperGrok plan, a narrow trial on *non-sensitive* operational chores (public-web research, drafting from files you attach) is reasonable. Do not treat named Bots as isolation, and do not enable local-computer execution unless there is a specific reason.

Rakazo

Rakazo (Rakazo contributors 2026b) is an Apache-2.0, self-hosted platform that describes itself as an "open-source Grok Bot alternative." The complete core product is in that repository: a web app, an Electron desktop client, and an Expo mobile app talking to one API. You bring model credentials (through the Pi harness) and choose where the computer runs.

As of 26 August 2026 the GitHub listing showed on the order of 1,300 stars. The README marks the product as beta.

Each bot has a thread, memory, routines, and history. Bots can delegate to peer bots or to short-lived subagents. Integrations can come from Composio or Pipedream Connect, or from a user-installed MCP server, Treg endpoint, or OpenAPI document.

Computers you host

The important design split is the same one Grok Bot has, except Rakazo lets you choose the provider (Rakazo contributors 2026a):

- **Team Computer** (default): bots share browser sessions and tools. Per-bot folders organize work; they are **not** a security boundary.
- **Private Computer**: the whole workspace is that bot's home.
- **Providers**: Docker (local default), E2B, Daytona, and Box for remote desktops, plus a trusted "this machine" desktop provider.

Pi runs in the Rakazo API/worker process, not inside the sandbox. Screen operation needs a model that can use image tool results. The Electron app is a client of the same API; on first launch it asks whether bots should keep using Docker or run on this Mac as you. That local-computer provider is the least isolated option and is not for a public or shared server (Rakazo contributors 2026b).

Self-hosting is a long-running API, a Graphile Worker, Postgres, and a computer provider — not a static site (self-hosting guide).

> Useful to us? Yes, as the open implementation to watch
>
> Rakazo is the closest thing we found to Grok Bot that we could legally inspect, self-host, and point at our own models. That matches lab priorities (local or lab-hosted compute, model choice, no extra vendor computer) better than Grok Bot itself.
> It is still a beta product with a real operations burden (Postgres, a worker, sandbox images, encrypted connector secrets). Do not adopt it as a dependency of lab workflow today. If we ever need persistent sandboxed teammates for browser-and-shell chores that should not run on an xAI VM, this is the codebase to evaluate — and its Team-versus-Private computer split is the right mental model even if we never run the app.

OpenClaw

OpenClaw (OpenClaw Foundation 2026a) is a MIT-licensed, self-hosted **gateway** for a personal assistant that meets you in messaging apps. You run one Gateway process on your own machine or a server. The Control UI, CLI, and TUI talk to that Gateway. Channels include WhatsApp, Telegram, Slack, Discord, Google Chat, Signal, iMessage, and others. Companion apps can add voice, canvas, camera, and device-local actions.

It works with hosted and local model providers. Skills, tools, and plugins extend the assistant. The project is built for a **single operator**.

This is not a Grok Bot clone. There is no vendor cloud VM per teammate. The Gateway stays on the host. **Sandboxing is off by default**: tool execution for the main session runs on the host unless you set `agents.defaults.sandbox` (OpenClaw Foundation 2026b). The docs warn that inbound messages are untrusted input, and that you should read the security and sandboxing guides before exposing the Gateway or connecting other users.

> Useful to us? Only with sandboxing, and not as a Grok Bot stand-in
>
> OpenClaw is relevant if we want an always-on assistant reachable from Slack or Telegram on hardware we already operate. That is a messaging-and-gateway problem, not a “bot with its own computer” problem.

Default host-side tool execution is incompatible with how we treat untrusted prompts and shared channels. Anyone trying it should turn sandboxing on (`non-main` or `all`) before connecting a group channel. It does not replace Claude Code, and it will not automatically load `ai-config`.

Claude Cowork

Claude Cowork (Anthropic 2026) uses the same agentic architecture as Claude Code, inside Claude Desktop (with web and mobile access described as rolling out). You describe an outcome; Claude works across local files and connected tools and returns documents, spreadsheets, or organized folders. Claude in Chrome is the browser path. Sub-agents split parallel workstreams.

Cowork is on paid Claude plans. Connectors, skills, and plugins load from **Customize** on the claude.ai account at session start. Cowork **does not read** the Claude Code CLI's `~/.claude` directory. A skill that exists only in the lab's symlink install must be added again under Customize (Anthropic 2026).

That last point is the lab-config trap: Cowork is the same vendor family as our daily coding agent, but it is not the same install path.

Useful to us? Yes, for knowledge work beside the terminal

Cowork is the lowest-friction vendor option if the task is files, slides, or Chrome automation and the operator already pays for Claude. It is not a persistent cloud computer of its own in the Grok Bot sense, and it will not pick up `ai-config` for free. Use it as a desktop knowledge-work surface, not as a second copy of our coding-agent stack.

ChatGPT Work

ChatGPT Work (OpenAI 2026) is OpenAI's surface for delegating a task with a clear outcome (a brief, deck, analysis, recurring update, or file). Chat remains for short answers. On the desktop app, Work can use local files, apps, and the browser when those tools are available. A **Work locally** versus **Cloud** control chooses whether the run needs your computer or should continue after you close the app (OpenAI 2026).

Computer Use is a plugin on the ChatGPT desktop app for macOS and Windows with Work and Codex: screen recording and accessibility permissions, plus per-app approval. Codex stays the software-development view in the same desktop app; Work is the everyday-work view of similar agent machinery (OpenAI 2026).

Useful to us? Only inside an existing ChatGPT workspace

ChatGPT Work is the OpenAI analog of "hand it a job and come back." Cloud Work is the closest OpenAI match to Grok Bot's laptop-closed computer. Local Work and Codex Computer Use are closer to Cowork: they operate *your* machine under approvals.
None of this loads `ai-config`. For repository work we already document Codex separately (Codex pull-request reviews). Do not add ChatGPT Work as a lab-wide tool unless a project is already inside that workspace and the data-handling rules for that workspace are acceptable.

Comparison

"Relevance to us" is the bottom line and follows from the rows above it.

Dimension	Claude Code + `ai-config` (ours)	Grok Bot	Rakazo	OpenClaw	Claude Cowork	ChatGPT Work
What it is	Repository coding-agent stack	Named teammates on a vendor cloud computer	Self-hosted Grok Bot-style platform	Self-hosted messaging gateway	Desktop knowledge-work agent	Outcome-oriented ChatGPT agent
Computer	Local checkout or cloud coding environment	One user-scoped xAI VM, shared by all Bots	Docker / E2B / Daytona / Box / this machine	Host Gateway; optional tool sandbox	Local files; Chrome; cloud sessions on some plans	Local or Cloud Work; optional Computer Use
Isolation of named agents	Worktrees, permissions, hooks	Not a security boundary	Team Computer is not; Private Computer is	Per-agent workspace; sandbox off by default	Session and folder access you grant	Workspace and plugin permissions

Dimension	Claude Code + `ai-config` (ours)	Grok Bot	Rakazo	OpenClaw	Claude Cowork	ChatGPT Work
Model choice	Whatever that harness is configured for	xAI / Cursor product path	Bring your own via Pi	Hosted or local providers	Anthropic	OpenAI
License / lock-in	Reusable across our repos	Closed; Cursor account required	Apache-2.0	MIT	Anthropic product	OpenAI product
Loads `ai-config` / `~/.claude`	Yes, when installed	Cursor plugin yes; `~/.claude` / Claude hooks no	No	No	No (Customize on claude.ai only)	No
Linux desktop app	Yes (CLI)	No	Web client; Electron is a client of that API	Yes	Claude Desktop	Desktop app; Computer Use is macOS/Windows
Relevance to us	The baseline	Pass for lab workflow; optional narrow trial	Evaluate if we need this shape	Gateway only, sandboxed	Knowledge work beside the terminal	Only if the project already lives in ChatGPT

Recommendation

For the lab, as of August 2026:

- **Keep repository work in Claude Code plus `ai-config`.** Grok Bot can load the Cursor plugin from that repo (skills, user-global rules, and commands in `.cursor-plugin/plugin.json`). It still does not load Claude Code's `~/.claude` symlink install, `CLAUDE.md` `@imports`, or Claude `hooks.json`; Cursor Cloud uses `.cursor/hooks.json` instead. Rakazo, OpenClaw, Cowork, and ChatGPT Work do not load that config. None of these is a substitute for a reviewable pull request.

- **Do not adopt Grok Bot as lab infrastructure.** It is a vendor cloud computer that shares logins across Bots, requires Cursor data storage, and has no Linux desktop app. A personal trial on non-sensitive chores is the most it should be.
- **Treat Rakazo as the open reference implementation** of the Grok Bot shape. Evaluate it if we need persistent sandboxed teammates under our keys; do not deploy it as production lab tooling while it is beta.
- **Do not confuse OpenClaw with Grok Bot.** It is a self-hosted messaging gateway. Sandbox it before any shared channel, or skip it.
- **Use Claude Cowork** when the job is local files or Chrome and you already pay for Claude, knowing you must re-add skills under Customize.
- **Leave ChatGPT Work** to projects that already use ChatGPT; keep coding in Codex.

None of these replaces the coding-agent platforms in Coding-Agent Platforms. They are teammates-with-computers and knowledge-work desktops. The portable lab config still lives in `ai-config`, and the review surface is still GitHub.

References

Anthropic. 2026. *Claude Cowork Overview*. Documentation. https://claude.com/docs/cowork/overview.

OpenAI. 2026. *Get Started with ChatGPT Work*. Documentation. https://learn.chatgpt.com/docs/get-started-with-work.

OpenClaw Foundation. 2026a. *OpenClaw*. Software. https://github.com/openclaw/openclaw.

OpenClaw Foundation. 2026b. *OpenClaw Sandboxing*. Documentation. https://docs.openclaw.ai/gateway/sandboxing.

Rakazo contributors. 2026a. *Rakazo Computer Runtime*. Documentation. https://github.com/elie222/rakazo/blob/main/docs/computer-runtime.md.

Rakazo contributors. 2026b. *Rakazo: Open-Source Grok Bot Alternative*. Software. https://github.com/elie222/rakazo.

xAI. 2026a. *Approvals, Security, and Privacy*. Documentation. https://docs.x.ai/grok-bot/approvals-security-and-privacy.

xAI. 2026b. *Get Started with Grok Bot*. Documentation. https://docs.x.ai/grok-bot/get-started.

xAI. 2026c. *Grok Bot Overview*. Documentation. https://docs.x.ai/grok-bot/overview.

xAI. 2026d. *Introducing Grok Bot*. Product announcement. https://x.ai/news/introducing-grok-bot.

xAI. 2026e. *Skills and Routines*. Documentation. https://docs.x.ai/grok-bot/skills-routines-and-automations.

xAI. 2026f. *Use the Computer and Apps*. Documentation. https://docs.x.ai/grok-bot/computer-and-apps.